Running this game (experimenter inputs):

- fa_outcome (default is 0): specify the penalty (number of steps back) for a false alarm (e.g., enter 5 if you want the FA penalty to be 5 steps back). Entering "beginning" means that the a false alarm resets the number of steps to 0.
- hit_outcome (default is 1): specify the reward (number of steps forward) for a hit (e.g., enter 5 if you want the Hit reward to be 5 steps forward).
- hit_goal (default is 20): specify the number of accumulated hits required to pass the game.

Lab 4 - Red Light, Green Light, Approaching the Doll

Last week, we built the Red Light, Green Light game, which is essentially a "Squid Game" themed Go/No-Go task.

To follow along today, please download the Lab 4 folder from CCLE. We will be starting with the **redlight_greenlight_approach_class_demo.psyexp** file, and modifying it as we work through the worksheet. Let's run this program and take a look at what we have so far...

Today, we will add some embellishments to make this game more interesting, both in terms of the research question that we may want to ask and in terms of making it more gamified and engaging for the participant. I'm calling this version "Red Light, Green Light, Approaching the Doll" for now (but please let me know if you can come up with a more creative name!).

Red Light, Green Light, Approaching the Doll

In this version of the Red Light, Green Light game, participants take a "step" closer to the finish line every time they make a Hit (i.e., pressing space when the back of the doll is facing them). If participants take a "step" closer when the doll turns around (i.e., False Alarming or pressing space when the doll is facing them), they need to take a few "steps" back. To give the sense of progress, the doll will appear larger with each "step". Once the participant reaches the target number of Hits, the session finishes.

To-Do List:

- Set a number of "steps" to get to the finish line
- With each "step", make the doll appear larger
- Introduce rules for each Hit and False Alarm (e.g., each Hit is 1 step closer, but each FA is 2 steps back)
- Enable experimenter to specify parameters at the start of the experiment
- Fun embellishments (if time)!

The code so far

Last week, we went over a few approaches to creating the sequence of images to be presented in each trial. However, there's a more programmatic approach to building these sequences. The advantage in the following code is that you can create a sequence of X length just by specifying the number of go trials (`num_0s`) and the number of no-go trials (`num_1s`).

```
# Programmatic approach
num_0s = 20
num_1s = 10

sequence = []
for i in range(num_0s):
    sequence.append(0)
for i in range(num_1s):
    sequence.append(1)

print(sequence)
```

```
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
```

With this approach, it's necessary to randomly shuffle the image sequence. You'll need to import the random module to do this.

```
# Randomize
import random
random.seed(101)   # seeding with "subject id"
random.shuffle(sequence)

print(sequence)
```

```
[0, 1, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 0]
```

A quick note on (pseudo)-randomization:

Randomization can be a tricky topic, so I want to point out some "best practice". Usually, it's a good idea to **seed** the randomization functions with a number that is unique to your participant (e.g., the subject ID). If using the random module in Python, this isn't absolutely necessary because they use the OS time as the default seed. However, it is necessary in Matlab and JS, and there are cases where studies were run without proper trial randomization because they forgot to seed the random functions.

Additionally, seeding is also useful when you want to replicate the "pseudorandom" sequence. For example, you might want to use the same seed across multiple sessions if you want the same "pseudorandom" sequence to be given to the same participant.

Make the doll image larger or smaller with each trial

The goal with this is to give a sense of movement when the doll appears larger (approaching) and when the doll appears smaller (retreating). Moreover, this gives the participant a sense of progress towards their target "distance".

Steps:

1. **In Begin Experiment tab**, create a `distance_goal` variable where we specify the number of hits or "steps" needed to pass the game. Also create a `distance_counter` variable that keeps track of the number of hits.
2. Create the `min_w`, `min_h`, `max_w`, `max_h` variables so that we can specify the starting and ending dimensions of the image.
3. Calculate `inc_w` and `inc_h` using the previous variables. These are the increments (in image dimensions) to be added each time a hit is recorded (i.e., this is like how "large" of a step to take). Calculated as: `inc_w = (max_w - min_w)/distance_goal`.

4. **In Begin Routine tab**, Update the `im_w` and `im_h` variables, which are the *current* dimensions of the doll image. These variables are calculated as: `im_w = min_w + distance_counter*inc_w`.
5. **In End Routine tab**, Here, we need to update the distance counter by 1 if a hit occurred.
6. We also want to check if `distance_counter == distance_goal`. When this conditional is True, terminate the trial early using this: `trials.finished = True`.
7. **In doll_im Properties**, In the Layout tab, find the "Size" field and enter `[im_w, im_h]`. Make sure to change the field to set every repeat!

Now test the code and see if the doll gets bigger!

Reward and Penalty

Our research objective is to shift our participant's response criterion by manipulating the reward/penalty outcomes of Hits and False Alarms. Reward is operationalized by pairing Hits with a number of steps forward, and penalty is operationalized by pairing False Alarms with a number of steps backward.

We want to include several different reward/penalty conditions. For the first condition, a Hit moves the subject 1 step forward and a False Alarm moves the subject 1 step back. This is essentially our "control" or "baseline" condition.

Steps:

1. **In End Routine tab**, when a False Alarm occurs (i.e., `sequence[trials.thisRepN] == 1 and task_kb.keys == 'space'`), subtract one step from the distance counter.
2. **Edge Case!** What if `distance_counter` is 0 and a False Alarm occurs? Should it be set to -1? We should handle this case by checking that `distance_counter` is greater than 0 before subtracting 1 from it.

User Defines the Reward and Penalty

In addition to the first condition, we may want to allow the experimenter to specify rules at the beginning of the session. To enable this, we will make use of the **Experiment info** attributes that can be set in the Experiment Settings window.

Steps:

1. **In Experiment Settings**, add 5 additional fields (and their defaults) to 'Experiment info': `hit_outcome` (1), `fa_outcome` (0), `hits_goal` (20), `num_go_trials` (200), `num_nogo_trials` (100)
2. **In Begin Experiment**, replace the assigned values of `num_0s`, `num_1s`, and `distance_goal` with `int(expInfo['num_go_trials'])`, `int(expInfo['num_nogo_trials'])`, and `int(expInfo['hits_goal'])`, respectively.
3. **In End Routine**, replace the value to update distance counter with `int(expInfo['hit_outcome'])` and `int(expInfo['fa_outcome'])`.
4. **Edge Case!** What happens if the `distance_counter` is less than the `fa_outcome`? `distance_counter` would be negative. We need to handle these cases.

Question: Let's say that our study consists of 2 group conditions. For one group, a False Alarm moves you 5 steps back. For a second group, a False Alarm moves you 10 steps back. Which group would you expect to False Alarm more (i.e., which group would have their decision criterion more biased towards the noise distribution)?

Now let's test our experiment! At the prompt, set `hit_outcome` to 1 and `fa_outcome` to 10. This means that for every hit, the subject moves forward one step. For everay false alarm, the subject moves back 10 steps.

As you can see, this flexibility in allowing the user to specify parameters is one of the advantages of taking a more programmatic approach earlier (instead of hard-coding everything).

Special Reward/Penalty Condition

Let's create one special reward/penalty condition where subjects are *returned to the beginning* if they make a False Alarm. The experimenter should be able to specify this conditions by typing "beginning" in the `fa_outcome` experiment info field.

Steps:

1. **In End Routine**, in the code block of the if statement that checks for False Alarms, add an additional nested code block that tests for whether the `expInfo['fa_outcome'] == "beginning"` . If it does, set `distance_counter` to 0; else, do what was done earlier.
2. Since the last step was difficult to articulate in words, I'll just explain the full code:

```
# update distance counter if trial
if sequence[trials.thisRepN] == 0 and task_kb.keys == 'space':      # test for HIT
    distance_counter = distance_counter + int(expInfo['hit_outcome'])
elif sequence[trials.thisRepN] == 1 and task_kb.keys == 'space':    # test for FA
    if expInfo['fa_outcome'] == "beginning":
        distance_counter = 0
    else:
        if distance_counter >= int(expInfo['fa_outcome']):
            distance_counter = distance_counter - int(expInfo['fa_outcome'])
        else:
            distance_counter = 0
```

Embellishments!

Now let's add some features to make this experiment feel more gamified*. Here are a couple of ideas:

- Add background music that continuously loops. Maybe a song from the Squid Game soundtrack?
- "Game Over" screen whenever you false alarm. For example, maybe have the doll's eye turn red and screen turn red?
- "Finished" screen when you complete the experiment. One idea is to have some ominous-sounding text like "Congratulations on passing the first game. Next game..." and then show the Squid Game dalgona symbols.
- Any other easy to execute ideas?

I've curated the stimuli to make these work, but I haven't coded them yet so I don't have instructions.

*Note that in a cognitive psychology lab experiment, there's probably no need to "gamify" the study. This is just for fun!

Next Week...

We will talk about data!

Lab 4 Homework - Red Light, Green Light, Approaching the Doll

This homework assignment will guide you through the steps of creating a "Congratulations" animation when your participant completes the experiment. We will be using several components and some code that we learned in class to make this animation. These problems are more like step-by-step instructions, but the goal is to help you gain some hands-on familiarity with GUI elements and the coding used to manipulate GUI components.

Please download the **entire lab4_hw folder (including sub-folders)** from Week 4 of CCLE. You will be doing this homework assignment by modifying the **redlight_greenlight_casino_{4MACS/4WINDOWS}.psyexp** file. The **resources** folder contain stimuli that are accessed in the experiment. There are two versions, redlight_greenlight_casino_4MACS.psyexp and redlight_greenlight_casino_4WINDOWS.psyexp. You will be using the one

that corresponds to the OS you're working on. Macs and Windows use different file path separators (forward slash for Macs and back slash for Windows). In the instructions, I've specified where Mac users should use forward slash in the file name and Windows users should use backslash.

You do not need to submit any other files - just submit your psychopy file named as **redlight_greenlight_casino_{yourname}.psyexp**.

If anything in the instructions is confusing or if you're encountering annoying bugs, please email me and I'd be more than happy to clarify!

The Objective

Our objective is to create a "congratulations" screen. To overcomplicate things, we will use a loop and some conditional statements to create a casino-like animation sequence that cycles through each of the three dalgona shapes from Squid Game. We'll also add some "casino" background sound to top it off!

Problem 1:

We're going to first add all the components and set up their properties. Please follow these steps and copy the values to the fields of the Property window for each component.

1. Insert a new routine called `congratulations` after the `trials` loop.
2. Add an image component called `circle` . In the `circle` Properties window, enter the following values to these field:
    - (Basic) Stop: 0.1
    - (Basic) Image: resources\dalgona_circle.png (**Windows**); resources/dalgona_circle.png (**Mac**)
    - (Layout) Size: (0.3, 0.3)
    - (Layout) Position: (-0.5,0)
    - (Appearance) Opacity: opacity_circle (change constant to **set every repeat**)
3. Add another image component called `star` with these properties:
    - (Basic) Stop: 0.1
    - *(Basic) Image: resources|dalgona_star.png* (**Windows**); resources/dalgona_star.png (**Mac**)
    - (Layout) Size: (0.3, 0.3)
    - *(Layout) Position: (0,0)*
    - *(Appearance) Opacity: opacity_star (change constant to **set every repeat**)*
4. Add another image component called `triangle` with these properties:
    - (Basic) Stop: 0.1
    - *(Basic) Image: resources|dalgona_triangle.png* (**Windows**); resources/dalgona_trianglee.png (**Mac**)
    - (Layout) Size: (0.3, 0.3)
    - *(Layout) Position: (0.5,0)*
    - *(Appearance) Opacity: opacity_triangle (change constant to **set every repeat**)*
5. Add a text component called `next_game` with these properties:
    - (Basic) Stop: 0.1
    - (Basic) Text:

      "Congratulations. You have passed the first game.

      Please select a shape for the next game..."

    - (Layout) Position: (0, 0.3)

- (Formatting) Letter Height: .04

6. Add a loop called `congrats_loop` around the `congratulations` routine.

   - nReps: congrats_nReps

Problem 2:

Now, we want to create code that updates the opacity of the circle, star, and triangle images with each iteration.

1. We want to randomly the number of repitions in this loop. First, add a new code component. Recall that in the `congrats_loop` properties, we pass in a variable called `congrats_nReps` . In the Begin Experiment tab, copy the following code:

```
# Randomly assign number of iteration.
congrats_nReps = random.randrange(60,63)
```

2. Next, we want to specify the opacity of the shapes, changing the opacity as the loop iterates. To do so, we make use of the modulus operator (that we learned about in lab 1! as a review, the modulus returns the **remainder** of a value divided by another value). Because we have three shapes, we want to set the opacities of the three shape sequentially, according to the current iteration of the loop (i.e., `thisRepN`). Copy and paste this code to the Begin Routine tab of your code component:

```
# Define the opacities of the 3 shapes depending on trial num
if congrats_loop.thisRepN % 3 == 0:
    opacities = [0.5, 0.5, 1]
elif congrats_loop.thisRepN % 3 == 1:
    opacities = [0.5, 1, 0.5]
elif congrats_loop.thisRepN % 3 == 2:
    opacities = [1, 0.5, 0.5]

# Assign the opacity variable to its respective values
opacity_circle = opacities[0]
opacity_star = opacities[1]
opacity_triangle = opacities[2]
```

At this point, you can test your code to see how it works! **Tip:** when prompted, set the `hit_goal` field to 1, so that you can skip the bulk of the experiment and go straight to the `congratulations` routine as soon as you get 1 hit.

Problem 3:

Now that we have the `congratulations` routine cycling through each shape, we want to make a duplicate routine that persists longer. We want the final selected shape to be presented on the screen for several seconds instead of just 0.5 s.

1. To do this, we need to copy and paste the `congratulations` routine. Make sure the `congratulations` routine is selected. Then, go to "Experiment -> Copy Routine". After, go to "Experiment -> Paste Routine". A window should pop up asking you to name this new duplicate routine, call it `congratulations_static` . Now, in this `congratulations_static` routine, you should have duplicate components from the `congratulations` routine.
2. Delete the code component. Right click the component and select "remove"
3. Add a Keyboard component (call it whatever). Make sure Force end of Routine is checked and set Allowed keys to 'space'.
4. Now, go through each of the components of the routine and set the **Stop** duration to 20.

That's it! Now after the "casino" animation sequence, you have a routine that persists longer on the shape that was randomly selected.

Problem 4:

As a final touch, let's add some "casino" sounds to the background. In the resources folder, there is a "casino_sound.wav" file.

Because we only want to play this once at the first iteration of the `congrats_loop`, we need to encapsulate the code that plays the casino sound in a conditional statement that checks for whether the current `thisRepN` is 0. Simply copy this code to the "Begin Routine" tab of your `congratulations` routine.

Copy this code if you're using WINDOWS:

```
# WINDOWS version: Audio set up and play (only if on the first trial)
if congrats_loop.thisRepN == 0:
    casino_sound = sound.Sound('resources\casino_sound.wav')
    casino_sound.play()
```

Copy this if you're using MAC:

```
# MAC version: Audio set up and play (only if on the first trial)
if congrats_loop.thisRepN == 0:
    casino_sound = sound.Sound('resources/casino_sound.wav')
    casino_sound.play()
```

Now, run your program and see what shape you get!